

PART 1 - DQC FRAMEWORK DESIGN

Pula Advisors Technical Assessment | May 2025

1. The DQC Framework: End to End

Data quality control is not a single step at the end of a project. It is a process that runs from before the first survey question is written all the way through to the final dataset handover. The earlier a problem is caught, the cheaper it is to fix. A bad survey question redesigned before launch costs nothing. The same issue discovered after 2,000 records have been collected can invalidate an entire field of analysis.

I structure the framework in five phases, each with defined activities, checks, and owners.

Phase	When	What Gets Checked	Who Does It
1. Design	Before collection starts	Survey questions reviewed, field validation rules configured in the app, skip logic tested, enumerator training completed	Data Analyst + Project Manager
2. Collection	While data is being collected	Daily completeness checks, submission speed flags, GPS plausibility, enumerator error rates monitored	Field Supervisors + Analyst
3. Audit	After full dataset received	Completeness profiling, cross-field logic checks, outlier detection, duplicate scan, DQC issue log built	Data Analyst
4. Fix	After audit complete	Issues triaged by severity, field team callbacks for critical records, corrections applied and documented	Analyst + Field Team
5. Sign-off	Before client handover	All critical issues resolved, assumptions documented, final confidence rating agreed	Data Manager + Client

Documenting Issues

Every issue found in Phase 3 goes into a single DQC log. Each entry records what was found, how many records are affected, the likely root cause, a severity rating of Critical, High, Medium, or Low, and the recommended fix. The severity rating is what drives prioritisation. Not everything is urgent and treating it all the same wastes time on low-impact issues while critical ones wait.

2. What to Catch Before vs After Collection

Problems split cleanly into two categories based on when they can be addressed. Pre-collection issues are ones you design against before a single enumerator enters the field. Post-collection issues are what your audit catches once the data is in. Both matter equally. Ignoring the first category means more of the second.

Before Collection: Design Against These

Issue to Design Against	What Goes Wrong if Ignored	How to Prevent It
Variety fields collected as bands (0-25%, 25-50%) not numbers	Cannot validate that Arabica and Robusta sum to 100%. Cannot compute averages or standard deviations. This happened in the X dataset.	Redesign as numeric fields with a mandatory sum-to-100% validation rule built into the survey tool
No mandatory field rules on critical columns	GPS coordinates, consent forms, and production figures get skipped. 20.7% of GPS records are missing in the X dataset.	Configure the survey tool to enforce required fields before a submission can be completed
District name collected as free text	"Nakeseke" appears as a phantom fifth district for 32 records. No such place exists in Uganda.	Use a fixed dropdown list. Never allow free text for any geography field
No range checks built into the collection tool	Enumerators can enter 28 acres for a smallholder farm with no system warning. Implausible values enter the dataset unchallenged.	Set expected bounds in the tool: farm size 0.1 to 15 acres, trees 50 to 5,000
No enumerator ID field in the survey	Cannot trace quality issues back to specific field staff. Fraud or systematic errors cannot be attributed or investigated.	Add enumerator ID as a mandatory auto-populated hidden field in every submission

After Collection: What the Audit Catches

The five issues below were found in the actual X Uganda dataset. They illustrate what a post-collection audit surfaces when pre-collection safeguards are missing.

Issue Found in the X Dataset	Why It Matters	What to Do
318 records (13.4%) where productive trees exceed total tree estimate	Logically impossible. Corrupts all tree density and productivity calculations across every district.	Flag all 318. Request enumerator callbacks for a sample. Exclude from density analysis until resolved.
520 records (21.9%) missing all 2022 production data	Cannot make reliable year-on-year comparisons for these farmers. Trend analysis is unreliable.	Classify as likely new 2023 enrollees. Document the assumption clearly. Exclude from trend analysis.
466 records (19.6%) with implausible tree density	Extreme outliers below 100 or above 2,500 trees per acre distort district averages significantly.	Flag records. Exclude from density averages with a documented rationale in the analysis.
445 records with no consent form	These records cannot be used in client reporting or	Escalate to project manager immediately. Do not include in

Issue Found in the X Dataset	Why It Matters	What to Do
	shared externally. This is a legal and ethical issue.	any client deliverable until resolved.
20 records with a coop name but farmer says not a member	Contradictory fields. Enumerator filled the name regardless of the membership answer.	Null the name for these 20 records. Prioritise the binary yes or no membership field.

3. Detecting Fabricated Data

Enumerators paid per submission have an incentive to fabricate records when field conditions are difficult or when daily targets are hard to meet. Fabricated data does not look random. It follows patterns that are detectable if you know what to look for. I screen for five types of signals.

Signal	What It Looks Like	How to Detect It
Data too clean	Fabricated records cluster around round numbers. Farms consistently reported as exactly 2 acres. Tree counts always in multiples of 100.	Calculate standard deviation of farm size and tree counts per enumerator. Flag anyone two or more standard deviations below the group average in variance.
Submissions too fast	Records completed in under 8 minutes. A genuine interview with GPS, consent, and 60+ questions takes at least 15 to 20 minutes.	Compute time between consecutive submissions per enumerator. Flag anyone with a consistent pattern below the minimum plausible interview duration.
GPS does not match district	A farmer claims Kayunga but GPS places them in a different region. Multiple records share identical coordinates.	Cross-check GPS against official Uganda district boundary shapefiles. Flag any coordinate falling outside the declared district polygon.
Records fail logic checks	Productive trees exceed total estimate. Both varieties at 75-100%. Coop name filled for a non-member. Fabricators do not think through field relationships.	Run the same cross-field validation rules used in the DQC audit. High failure rates from a single enumerator are a fraud signal not a data quality issue.
Near-duplicate submissions	Same farmer submitted twice with minor variations in farm size or GPS to avoid exact duplicate detection.	Run fuzzy matching on GPS coordinates, farm size, and household member count. Flag enumerators with high near-duplicate rates.

Escalation Process

A single anomaly is a data entry error. Three or more flags on one enumerator is a pattern that needs investigation. My escalation threshold is three concurrent signals.

- Step 1 Document the flags with supporting evidence: record IDs, timestamps, GPS coordinates.
- Step 2 Raise with the field supervisor without naming the enumerator. Ask if field conditions explain the pattern.
- Step 3 If the explanation does not hold, escalate to the project manager with a written summary.

- Step 4 Verify a sample of their records (minimum 20%) by phone or re-visit. Confirmed fabrication means removal of those records and the enumerator.
- Step 5 Document the outcome in the master log. The client must be informed of any records removed and why.

4. Applying the Framework to the X Uganda Dataset

The following observations come from a structured first review of the X dataset before any analysis was run. Five issues stood out immediately. Each one has a direct first action.

What I Noticed	Records Affected	Why It Concerns Me	First Check to Run
Consent form missing	445 (18.7%)	Legal and ethical blocker. Cannot go to a client without resolution.	Separate consented and unconsented records before any other work.
GPS data missing	491 (20.7%)	Cannot verify farm locations or detect any geospatial fraud signals.	Cross-check missing GPS against enumerator ID. Investigate if specific staff skipped it.
Productive trees exceed total tree estimate	318 (13.4%)	Logically impossible. Corrupts density and productivity analysis across all districts.	Flag all 318. Cross-field rule: productive trees must be less than or equal to total estimate.
All 2022 production data missing	520 (21.9%)	Makes year-on-year comparisons unreliable for over a fifth of the dataset.	Classify as new farmers. Document assumption. Exclude from trend analysis.
District name typo Nakeseke vs Nakaseke	32 (1.3%)	Creates a phantom fifth district. Signals free-text was used where a dropdown was needed.	Recode all 32 records to Nakaseke. Enforce dropdown before next collection round.

Overall Confidence Rating: MEDIUM

The dataset is usable for 2023 cross-sectional analysis with documented assumptions. Core fields including Farmer ID, district, farm size, and 2023 FAQ production are above 99% complete, which is a strong base. However, three things reduce confidence: 18.7% of records have no consent flag which is a legal blocker, 21.9% cannot be used for year-on-year comparisons, and 13.4% contain a logical impossibility in tree counts. The dataset is not ready for trend analysis or fraud-free sign-off without resolving these first.

5. Scaling to 50,000 Farmers: From Manual Checks to a Data Pipeline

At 500 farmers the approach used in this assessment works well. Loading data into DuckDB, running SQL validation views, and generating charts is transparent, repeatable, and leaves a clear audit trail. At 5,000 farmers it starts to strain. At 50,000 it breaks entirely.

The same analyst cannot manually review 50,000 records. Excel becomes slow and error-prone. DQC checks that ran once at the end of collection need to run continuously during it. By the time problems are found in a manual process, thousands of records are already affected and field teams have moved on.

The solution is not a different analyst. It is a different infrastructure. I recommend a dbt-based data pipeline that automates every step the analyst currently does by hand and adds real-time alerting so problems surface within minutes of each submission batch landing.

Recommended Pipeline Architecture

Layer	Tool	What It Does	DQC Role at This Layer
Ingestion	ODK / KoBoToolbox + Python	Collects raw survey submissions and pushes to the central database on completion.	Field validation fires at point of entry. Mandatory fields enforced. Range checks block implausible values before submission lands.
Raw Layer	PostgreSQL / BigQuery	Stores every record exactly as received. Nothing is deleted or changed here.	Immutable audit trail. Any transformation error can be traced back to the original submission at any time.
Staging (dbt)	dbt staging models	Cleans and renames columns. Fixes known typos. Standardises yes or no fields. Drops blank columns.	First transformation layer. All cleaning logic is in version-controlled SQL. Every change is documented and reproducible.
Intermediate (dbt)	dbt intermediate models	Derives calculated fields: trees per acre, year-on-year production change, DQC flags.	This is where the validation rules live as SQL CASE statements. Every flag is a dedicated column visible to all downstream users.
Marts (dbt)	dbt mart models	Final analytical tables: production by district, labour compliance, cooperative comparison.	Only validated, flagged records reach the mart layer. Analysts query marts, never raw tables.
Tests (dbt)	dbt built-in and custom tests	Automated tests run on every pipeline trigger. A failed test blocks downstream models.	Tests enforce: not_null on required fields, accepted_values on yes or no fields, custom SQL for cross-field logic rules.
Alerts	Slack or email via dbt Cloud	Notifies the data team when any test fails or a completeness threshold is breached.	Real-time DQC. A consent gap or GPS spike in a new batch is flagged within minutes, not weeks after collection ends.
Reporting	Metabase or Power BI	Live dashboards on quality metrics and analytical	DQC dashboard shows completion rates, flag

Layer	Tool	What It Does	DQC Role at This Layer
		outputs updated with every pipeline run.	counts, and enumerator error rates in real time.

Why dbt specifically?

dbt (data build tool) treats SQL transformations as code. Every model is version-controlled in Git, every test is documented, and every run produces a full log showing which models passed or failed and why. DQC becomes an automated process that runs every time new data arrives rather than something one analyst does manually at the end of a project. The staging, intermediate, and mart layer structure used in this assessment already mirrors the dbt pattern exactly.

What the dbt Tests Look Like

dbt supports both built-in tests for common checks and custom SQL tests for business-specific logic. The validation rules from this assessment translate directly.

Test Type	What It Checks	Example
not_null	Flags any required field that is empty	columns: - name: farmer_id tests: - not_null
accepted_values	Ensures yes or no fields only contain valid responses	- name: underage_workers tests: - accepted_values: values: ['yes','no']
Custom: tree logic	Productive trees cannot exceed the total tree estimate	SELECT farmer_id FROM stg_farmers WHERE productive_trees > estimated_trees
Custom: density	Flags implausible trees per acre outside 100 to 2,500 range	SELECT farmer_id FROM int_enriched WHERE trees_per_acre < 100 OR trees_per_acre > 2500
Custom: variety	Both varieties cannot both be at 75-100% on one farm	SELECT farmer_id FROM stg_farmers WHERE pct_arabica = '75-100%' AND pct_robusta = '75-100%'
Freshness check	Alerts if no new data has arrived within the expected window	sources: - name: raw_farmers freshness: warn_after: {count: 1, period: day}

Every time the pipeline runs, all tests execute automatically. A failed test can be configured to block the entire downstream pipeline, preventing bad data from reaching the reporting layer without anyone needing to manually intervene.

Real-Time Alerts

The key difference between a manual DQC process and a pipeline-based one is when the team finds out about problems. In a manual process, problems are found at the end of collection. In a pipeline, the team knows within minutes.

- o dbt Cloud or Airflow runs the pipeline on a schedule or triggered by each new submission batch.
- o If any test fails, a Slack message or email fires immediately with the test name, number of records affected, and a link to the full log.

- o Threshold alerts fire separately: for example, alert if consent form completion drops below 90% in the latest batch.
- o Enumerator scorecards are generated automatically each morning showing error rates, submission speeds, and GPS anomalies per field agent.

Validating Against Existing Farmer Profiles

Once farmers have been profiled in a previous season, their records become the benchmark for all future submissions. A second survey round is not just adding new rows. It is updating an existing profile, which enables a much richer set of validations that are only possible when historical data exists.

Validation Check	Logic Applied	Action When Flag Fires
Is this a returning farmer?	Join incoming record to existing profile by farmer_id. Match found means return visit.	Apply stricter validations. Farm size, district, and GPS should not change dramatically between seasons.
Farm size drift	Flag if new farm size differs from the historical profile by more than 30%.	Hold in a review queue. Alert field supervisor to verify with the farmer directly.
Production cliff	Flag if 2023 production is more than 3x or less than 0.2x the farmer historical average.	Flag for analyst review. Could be genuine (weather, disease) or a data entry error. Requires a judgment call.
GPS drift	Flag if new GPS is more than 500 metres from the farmer historical GPS centroid.	Likely a wrong GPS capture. Send for re-verification before record is accepted.
New cooperative or buyer	If a returning farmer now reports a cooperative or buyer not in their profile, flag as a change event.	Log as a profile update. Trigger a data enrichment step rather than treating it as an error.
Duplicate submission	Check if farmer_id already has a 2023 submission. If yes, new record is a duplicate.	Block the duplicate. Alert field supervisor. Keep original unless a verified correction is submitted.

How this works in the pipeline

The dbt intermediate layer joins every incoming record against the existing farmer profile table using farmer_id. If the join succeeds, the record is a returning farmer and drift checks apply automatically. If no match is found, it is treated as a new enrollment. A single LEFT JOIN in the staging model handles this with no manual lookup needed.

Integration with External Systems

As the farmer database grows, the pipeline connects to external systems to enrich and validate data automatically, removing work that currently requires manual cross-referencing.

External System	What It Provides	DQC Value
Uganda District Boundaries (GIS shapefile)	Official GPS polygon for every district	Auto-validates GPS coordinates against declared district. Flags any farmer whose GPS falls outside their stated district boundary.

External System	What It Provides	DQC Value
Internal CRM or ERP	Historical transaction and payment records	Cross-checks declared buyer relationships and production volumes against what X has actually purchased from each farmer in prior seasons.

How the Approach Changes as You Scale

Aspect	500 Farmers - Manual Approach	50,000 Farmers - Pipeline Approach
When problems are found	End of collection. Weeks after the issue first appeared.	Real time. Within minutes of each batch landing.
Validation logic	SQL views in DuckDB. Run manually by the analyst.	dbt tests in version-controlled SQL. Run automatically on every pipeline trigger.
Alerts	Analyst emails the team after reviewing the data.	Slack or email fires automatically when any test fails or any threshold is breached.
Returning farmer checks	Manual lookup against the previous season dataset.	Pipeline joins every record against the farmer profile table using farmer_id on arrival.
Enumerator monitoring	Analyst reviews submissions by hand at end of collection.	Daily scorecards generated automatically per field agent showing error rates and speed anomalies.
External validation	Manual cross-reference with external sources when time allows.	API integrations pull district boundaries, UCDA data, and weather context automatically on each run.
Audit trail	Excel files and Python scripts saved locally.	Every model run logged in dbt. Full data lineage from raw submission to final mart visible in dbt docs.